

DSAA3073: Theories in Data Science

Week 11: Deep Learning II: Neural Tangent Kernel & Implicit Regularization

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-27

Explorative projection

Background: Kernels and Reproducing Kernel Hilbert Spaces

Before diving into Neural Tangent Kernel theory, we need essential background on kernel methods. This section provides a self-contained introduction—no prior kernel knowledge required.

Kernels as Similarity Measures

Definition: Kernel Function

A **kernel** is a function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ that measures similarity between inputs. For a kernel to be valid, it must be:

1. **Symmetric:** $k(x, x') = k(x', x)$
2. **Positive semi-definite:** For any $x_1, \dots, x_n$ and $c_1, \dots, c_n$:

$$\sum_{i,j} c_i c_j k(x_i, x_j) \geq 0 \quad (1)$$

Key examples:

- Linear kernel: $k(x, x') = \langle x, x' \rangle$
- Polynomial kernel: $k(x, x') = (\langle x, x' \rangle + c)^d$
- RBF (Gaussian) kernel: $k(x, x') = \exp(-\gamma \|x - x'\|^2)$

Intuition: Kernels implicitly map inputs to a high-dimensional feature space where linear methods work well, without ever computing the mapping explicitly.

Reproducing Kernel Hilbert Spaces

Definition: Reproducing Kernel Hilbert Space (RKHS)

A **Reproducing Kernel Hilbert Space** $\mathcal{H}$ associated with kernel k is a Hilbert space of functions $f : \mathcal{X} \rightarrow \mathbb{R}$ with the **reproducing property**:

$$f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}} \quad \text{for all } f \in \mathcal{H}, x \in \mathcal{X} \quad (2)$$

where $k(x, \cdot)$ is the function $x' \mapsto k(x, x')$.

What this means: Evaluating a function f at point x is just an inner product in the RKHS! This seemingly abstract property has profound consequences.

Key properties:

1. Every positive semi-definite kernel uniquely defines an RKHS
2. Functions in the RKHS have a norm: $\|f\|_{\mathcal{H}}^2$
3. The **representer theorem**: solutions to regularized learning problems have form $f(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$

Kernel Ridge Regression

Example: Learning in RKHS

Ridge regression in an RKHS:

$$\min_{f \in \mathcal{H}} \left(\sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right) \quad (3)$$

Solution: $f^*(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$ where $\alpha = (K + \lambda I)^{-1} y$

Here K is the **kernel matrix**: $K_{ij} = k(x_i, x_j)$.

Why this works: The RKHS norm $\|f\|_{\mathcal{H}}^2$ penalizes function complexity. Kernel ridge regression balances fit (training error) against smoothness (RKHS norm)—the same bias-variance tradeoff from Week 4, now in function space.

Connection to Neural Networks

Why this matters for deep learning: Neural Tangent Kernel theory reveals that infinitely wide neural networks trained with gradient descent behave exactly like kernel ridge regression with a specific kernel (the NTK) determined by the network architecture. This connection:

1. Explains why overparameterized networks are easy to optimize
2. Connects deep learning to classical kernel theory
3. Provides generalization bounds via RKHS norm

The key insight: During training, an infinite-width network's predictions evolve according to kernel regression in the RKHS defined by its NTK. The network never really “learns features”—it stays close to initialization and acts like a fixed kernel method. This is the **lazy training regime**.

Neural Tangent Kernel Theory

This section connects neural networks to kernel methods from Weeks 6-8. The key insight: very wide neural networks behave almost like kernel methods.

Opening Challenge

For very wide networks trained with small learning rates, training dynamics are almost linear! This surprising connection links neural networks to kernel methods, providing theoretical guarantees for optimization.

Connection

Connection to Week 11 Background Section (RKHS): Recall that kernels measure similarity $k(x, x')$ between inputs and define a **reproducing kernel Hilbert space (RKHS)** where learning takes place. The Neural Tangent Kernel plays the same role for infinite-width networks, connecting deep learning to kernel theory.

Linearization and the NTK

Definition: Neural Tangent Kernel

The **Neural Tangent Kernel (NTK)** at initialization θ_0 is:

$$\Theta(x, x') = \nabla_{\theta} f(x; \theta_0)^T \nabla_{\theta} f(x'; \theta_0) = \sum_{i=1}^p \frac{\partial f(x; \theta_0)}{\partial \theta_i} \frac{\partial f(x'; \theta_0)}{\partial \theta_i} \quad (4)$$

This measures the alignment between parameter gradients at two inputs x and x' . Intuitively, if $\Theta(x, x')$ is large, changing parameters to fit x will also change the output at x' (the two inputs are similar from the network's perspective).

Example: NTK for a Linear Model

For the ordinary linear model $f_w(x) = w^T x$, the parameter gradient is $\nabla_w f_w(x) = x$. Therefore the NTK is

$$\Theta(x, x') = x^T x', \quad (5)$$

exactly the standard linear kernel. This is why parameter gradients can be viewed as features.

Example: NTK for Two-Layer Network

For a two-layer network $f(x) = \left(\frac{1}{\sqrt{m}}\right) \sum_{i=1}^m a_i \sigma(w_i^T x)$ with $a_i \in \{-1, +1\}$ fixed (only training the input weights w_i):

$$\Theta(x, x') = \frac{1}{m} \sum_{i=1}^m \sigma'(w_i^T x) \sigma'(w_i^T x') x^T x' \quad (6)$$

where σ' is the derivative of the activation function.

As width $m \rightarrow \infty$, by the **law of large numbers** (the average of many independent random variables converges to their expectation):

$$\Theta(x, x') \rightarrow \mathbb{E}_{w \sim \mathcal{N}(0, I)} [\sigma'(w^T x) \sigma'(w^T x')] x^T x' \quad (7)$$

The NTK becomes deterministic! The randomness from initialization averages out.

Note: Because a_i are fixed, only the input-weight gradients contribute. If output weights are also trained, the standard two-layer NTK has an additional covariance term from the a_i gradients.

Pause and Think

Question: Why does the NTK become deterministic as width increases?

Hint: Think about the law of large numbers. What happens to the average of many i.i.d. (independent and identically distributed) random variables?

Answer:

The NTK is an average over m neurons: $\Theta = \frac{1}{m} \sum_{i=1}^m$ contribution from neuron i . Each neuron is initialized independently with random weights. As $m \rightarrow \infty$, this average concentrates around its expectation by the law of large numbers (the sample average converges to the population mean), becoming deterministic. The randomness from initialization averages out.

Example: Computing NTK for ReLU

For ReLU activation $\sigma(z) = \max(0, z)$, the derivative is $\sigma'(z) = \mathbb{1}_{z>0}$ (the indicator function: 1 if $z > 0$, and 0 otherwise) away from $z = 0$. The value at $z = 0$ is irrelevant under continuous Gaussian initialization because it occurs with probability zero.

The NTK becomes:

$$\Theta(x, x') = \mathbb{E}_{w \sim \mathcal{N}(0, I)} [\mathbb{1}_{w^T x > 0} \mathbb{1}_{w^T x' > 0}] x^T x' \quad (8)$$

The expectation is the probability that both $w^T x$ and $w^T x'$ are positive (both projections land in the positive half-space):

$$\mathbb{E}[\mathbb{1}_{w^T x > 0} \mathbb{1}_{w^T x' > 0}] = \mathbb{P}(w^T x > 0 \text{ and } w^T x' > 0) \quad (9)$$

For Gaussian w , this depends on the angle θ between x and x' . Using geometric arguments from probability theory:

$$\mathbb{P}(w^T x > 0 \text{ and } w^T x' > 0) = \frac{\pi - \theta}{2\pi} = \frac{1}{2} - \frac{\theta}{2\pi} \quad (10)$$

where $\cos \theta = \frac{x^T x'}{\|x\|_2 \|x'\|_2}$ is the normalized dot product.

Therefore, the ReLU NTK has a closed form:

$$\Theta(x, x') = \left(\frac{\pi - \theta}{2\pi} \right) x^T x' = \left(\frac{1}{2} - \frac{\arccos\left(\frac{x^T x'}{\|x\|_2 \|x'\|_2}\right)}{2\pi} \right) x^T x' \quad (11)$$

This is a specific kernel determined entirely by the ReLU activation and Gaussian initialization—no learning involved!

🧠 Mental Model: NTK as Feature Learning Frozen at Initialization

Understanding the NTK regime:

Standard neural network training:

- Parameters θ change significantly during training
- Features $\varphi(x) = \nabla_{\theta} f(x; \theta)$ evolve (feature learning)
- Network output $f(x; \theta)$ is highly nonlinear in parameters

NTK regime (infinite width):

- Parameters change infinitesimally (each neuron's contribution vanishes as $\frac{1}{\sqrt{m}}$)
- Features $\varphi(x) \approx \nabla_{\theta} f(x; \theta_0)$ stay nearly constant
- Training becomes **linear** in features frozen at initialization
- Equivalent to kernel regression with kernel $\Theta(x, x') = \varphi(x)^T \varphi(x')$

Key insight: Infinite width “freezes” the features at random initialization. Training only adjusts coefficients, not features. This is why NTK connects to kernel methods (Week 11 Background Section)!

Limitation: Real networks (finite width) learn features. NTK explains optimization but misses representation learning.

Theorem: NTK Training Dynamics (Jacot et al. 2018; simplified)

Assume:

- **Architecture:** Fully-connected network with L layers and width m per hidden layer
- **Activation:** $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is Lipschitz continuous and bounded (e.g., tanh, sigmoid, or smoothed ReLU)
- **Initialization:** Weights $\theta_0 \sim \mathcal{N}(0, \sigma_w^2 I)$ with appropriate scaling (e.g., $\sigma_w = \frac{1}{\sqrt{m}}$)
- **Loss:** Squared loss $\mathcal{L}_{S(\theta)} = \frac{1}{2n} \sum_{i=1}^n (f(x_i; \theta) - y_i)^2$ on training set $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- **Training:** Gradient flow (continuous-time gradient descent) $\frac{d\theta}{dt} = -\nabla_{\theta} \mathcal{L}_{S(\theta_t)}$
- **Regime:** Infinite-width limit $m \rightarrow \infty$ with fixed dataset size n

Then: The network output dynamics become linear:

$$\frac{d\mathbf{f}_t}{dt} = -\frac{1}{n} \Theta(\mathbf{f}_t - \mathbf{y}) \quad (12)$$

where:

- $\mathbf{f}_t = (f(x_1; \theta_t), \dots, f(x_n; \theta_t))^T \in \mathbb{R}^n$ = network outputs on training data
- $\mathbf{y} = (y_1, \dots, y_n)^T \in \mathbb{R}^n$ = target values
- $\Theta \in \mathbb{R}^{n \times n}$ = NTK Gram matrix with $\Theta_{ij} = \Theta(x_i, x_j)$
- $\Theta(x, x') = \text{NTK kernel (constant in infinite-width limit)}$

This is a **linear** ODE, unlike the original nonlinear neural network dynamics.

Proof. Sketch of the key steps:

1. **Gradient descent update:**

$$\frac{d\theta_t}{dt} = -\nabla_{\theta} \mathcal{L}_{S(\theta_t)} = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \nabla_{\theta} f(x_i; \theta_t)$$

2. Network output evolution:

$$\begin{aligned} \frac{df(x; \theta_t)}{dt} &= \nabla_{\theta} f(x; \theta_t)^T \frac{d\theta_t}{dt} \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \nabla_{\theta} f(x; \theta_t)^T \nabla_{\theta} f(x_i; \theta_t) \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \Theta_t(x, x_i) \end{aligned} \quad (14)$$

When is Θ positive definite?

For non-degenerate data (no duplicates, sufficient diversity), the NTK Gram matrix is positive definite with high probability under random initialization. The minimum eigenvalue $\lambda_{\min}$ depends on data geometry and kernel properties.

3. **Infinite-width limit:** As width $m \rightarrow \infty$, on a fixed finite dataset and over suitable training horizons, the NTK remains approximately constant: $\Theta_t(x, x') \approx \Theta_0(x, x') = \Theta(x, x')$.
4. This gives the linear dynamics: $\frac{df_t}{dt} = -\frac{1}{n} \Theta(f_t - y)$.

■

Theorem: Exponential Convergence in NTK Regime

Assume:

- NTK training dynamics (from previous theorem): $\frac{df_t}{dt} = -\frac{1}{n} \Theta(f_t - y)$
- **Non-degenerate NTK:** Gram matrix $\Theta \in \mathbb{R}^{n \times n}$ is positive definite with smallest eigenvalue $\lambda_{\min}(\Theta) > 0$
- **Data condition:** Training inputs $\{x_1, \dots, x_n\}$ are distinct and non-degenerate (ensures positive definiteness)

Then: The training error decays exponentially:

$$\|f_t - y\|_2 \leq \exp\left(-\frac{\lambda_{\min} t}{n}\right) \|f_0 - y\|_2 \quad (15)$$

Interpretation: Convergence rate depends on $\frac{\lambda_{\min}}{n}$. Larger minimum eigenvalue $\rightarrow$ faster convergence. Training achieves ε -accuracy in $O\left(\frac{n}{\lambda_{\min}} \log\left(\frac{1}{\varepsilon}\right)\right)$ time.

Proof. The linear ODE $\frac{df_t}{dt} = -\frac{1}{n} \Theta(f_t - y)$ has solution:

$$f_t - y = \exp\left(-\frac{\Theta t}{n}\right) (f_0 - y) \quad (16)$$

Since Θ is positive definite:

$$\|f_t - y\|_2 \leq \left\| \exp\left(-\frac{\Theta t}{n}\right) \right\| \|f_0 - y\|_2 \leq \exp\left(-\frac{\lambda_{\min} t}{n}\right) \|f_0 - y\|_2 \quad (17)$$

■

Key Takeaway

In the infinite-width limit, neural network training behaves like **kernel gradient flow** or **kernel interpolation** in the NTK RKHS (reproducing kernel Hilbert space associated with kernel Θ); with explicit ridge regularization, the predictor corresponds to kernel ridge regression. Dynamics are linear under the constant-NTK approximation and converge exponentially when the empirical NTK Gram matrix Θ is positive definite (all eigenvalues positive).

Connection

Week 6-8 connection: The NTK plays the role of kernel $k(x, x')$ in kernel gradient flow/interpolation (the dynamics of continuously optimizing in the RKHS); with ridge regularization (adding $\lambda \|f\|_{\mathcal{H}}^2$ penalty), it connects to kernel ridge regression (introduced in Week 7). But unlike hand-designed kernels (polynomial, RBF), the NTK is determined by architecture and initialization—it's not chosen a priori but emerges from the network structure.

Sample complexity implications: In the NTK regime, sample complexity matches kernel methods: $\tilde{O}(\frac{1}{\varepsilon^2})$ samples for ε -accuracy under RKHS norm bounds (Week 7). This can be suboptimal compared to feature learning when the target function has hierarchical structure exploitable by deep representations—highlighting the limitation of lazy training.

✓ Checkpoint

NTK theory summary:

1. Infinite-width networks behave like kernel methods with kernel $\Theta(x, x')$ determined by architecture and initialization
2. Training dynamics become **linear** (gradient flow in RKHS, unlike the original non-linear neural network dynamics)
3. Convergence is exponential when the NTK Gram matrix is positive definite (like kernel ridge regression from Week 7)
4. NTK is determined by architecture and initialization, not learned during training (features are frozen at initialization)

Key limitation: NTK assumes features don't change during training (lazy training regime). Real finite-width networks learn features—this is their main advantage over kernel methods!

Lazy Training vs. Feature Learning

Definition: Training Regimes

Neural networks exhibit different behaviors depending on width:

- **Lazy training:** NTK stays approximately constant during training. The network behaves like a kernel method—features are frozen.
- **Feature learning:** NTK changes significantly during training. The network learns task-specific representations—features evolve during training.

Width requirement impractical

Original proof requires $m = \Omega(n^6)$ neurons—impractical! Recent work improves to $m = O(n^2)$ or $m = O(n)$ under stronger assumptions. The theorem shows convergence is possible in theory, not that it's necessary in practice.

Theorem: Global Convergence for Overparameterized Networks (Du et al. 2019; simplified)

Assume:

- **Architecture:** Two-layer network $f(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^T x)$ with m hidden units
- **Width:** $m \geq C \frac{n^6}{\delta^2 \lambda_0^2}$ for constant C (extremely overparameterized)
- **Initialization:** Weights $w_i \sim \mathcal{N}(0, I)$, second-layer weights $a_i \in \{-1, +1\}$ fixed
- **Activation:** ReLU or smooth activation with bounded derivatives
- **Data:** Training set $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ with $\|x_i\|_2 = 1$ (normalized)
- **Non-degeneracy:** Minimum eigenvalue $\lambda_0 = \lambda_{\min}(\Theta_0) > 0$ at initialization
- **Learning rate:** $\eta = O\left(\frac{\lambda_0}{n}\right)$ (sufficiently small)

Then: With probability $\geq 1 - \delta$ over random initialization, gradient descent converges to training loss $\mathcal{L}_{S(\theta_T)} < \varepsilon$ in $T = O\left(\frac{n}{\lambda_0} \log\left(\frac{1}{\varepsilon}\right)\right)$ iterations.

Key insight: Sufficient width ensures NTK stays nearly constant (lazy training), reducing to linear dynamics with guaranteed convergence.

Wrong Path: NTK Theory Explains All of Deep Learning

Why incomplete:

1. Assumes lazy training (networks stay near initialization), but practical networks learn features
2. Requires impractically large width
3. Explains optimization but not generalization
4. Treats all architectures similarly, but architecture matters enormously in practice
5. Ignores the power of feature learning—the main advantage of deep learning over kernels!

What NTK does explain:

- Why very wide networks optimize easily (lazy regime)
- Connection between neural networks and kernel methods
- Lower bound on what networks can achieve (kernels $\subseteq$ neural networks)

What NTK misses:

- Why finite-width networks generalize better than kernels
- How networks learn hierarchical representations
- Why architecture design matters

Connection

Preview of Week 12: NTK explains why networks **optimize** (lazy training $\rightarrow$ kernel methods $\rightarrow$ PL condition $\rightarrow$ exponential convergence). But why do they **generalize**? Week 12 covers implicit regularization, compression, and modern phenomena (grokking, scaling laws).

Mental Model: The Width Spectrum

Understanding training across different widths:

Narrow networks ($m = O(1)$):

- Hard to optimize (many bad local minima)
- If optimization succeeds: good generalization (strong implicit bias)
- High variance (sensitive to initialization)

Moderate width ($m = O(n)$ to $O(n^2)$):

- Easier optimization (fewer bad local minima)
- Feature learning (NTK changes during training)
- Best generalization (sweet spot)
- This is the practical regime!

Infinite width ($m \rightarrow \infty$):

- Easiest optimization (linear dynamics, exponential convergence)
- No feature learning (lazy training)
- Kernel method behavior
- Explains optimization but misses generalization benefits

Key insight: Finite width is crucial for both tractable optimization AND good generalization through feature learning!

This leads us to the next question: NTK theory explains optimization convergence, but we still face the generalization puzzle. In the overparameterized regime, there are infinitely many solutions that perfectly fit the training data—why does gradient descent find one that generalizes well? The answer lies in **implicit regularization**: gradient descent has built-in biases that favor simple, generalizable solutions even without explicit penalties. This is perhaps the deepest insight in modern deep learning theory.

Implicit Regularization

Having established that overparameterized networks can be optimized (previous sections), we now address why they generalize.

Opening Challenge

Networks generalize without explicit regularization (no manually added penalty terms). Why? The optimization algorithm itself—gradient descent—has an **implicit bias** (a preference for certain types of solutions that emerges from the dynamics, not from the loss function).

Connection

Connection to Week 2-3 (PAC Learning and VC Theory): Classical VC theory predicts sample complexity $O(\frac{d}{\epsilon^2})$ where d is VC dimension (a measure of hypothesis class complexity). For neural networks, d can exceed sample size n (e.g., $d = O(p \log p)$ where p is the number of parameters), yet networks generalize! Classical theory predicts overfitting but we observe good generalization—implicit regularization explains this discrepancy.

Mental Model: The Implicit Regularization Mystery

Consider two solutions to a learning problem:

Scenario: Binary classification, linearly separable data

- Many weight vectors w achieve zero training error
- Which one does gradient descent find?

Solution A: Arbitrary separator

- Small margin
- Sensitive to perturbations
- Poor generalization

Solution B: Max-margin separator (SVM)

- Large margin
- Robust to perturbations
- Good generalization

Mystery: We never told gradient descent to maximize margin, yet it finds solution B!

Resolution: The optimization algorithm (gradient descent) + loss function (logistic/exponential) + initialization (small weights) together create **implicit regularization**—a bias toward “simple” solutions.

This is the key to understanding why overparameterized networks generalize.

Implicit Bias in Linear Networks

Why factorization matters

Direct gradient descent on W would converge to minimum Frobenius norm $\|W\|_F$. Factorization $W = UV^T$ + gradient flow changes the implicit bias to nuclear norm $\|W\|_*$, which encourages low rank.

Theorem: Implicit Bias in Linear Networks (Gunasekar et al. 2018)

Gradient descent on linear networks often biases the solution towards low-rank predictors. In some two-factor matrix factorization settings (parameterizing $W = UV^T$ and optimizing over U, V instead of directly over W), this bias corresponds to **nuclear-norm minimization** (minimizing the sum of singular values):

$$W^* = \operatorname{argmin}\{\|W\|_* : Wx_i = y_i \text{ for all } i\}. \quad (18)$$

where $\|W\|_* = \sum_j \sigma_j(W)$ is the nuclear norm (sum of singular values). For deeper linear networks, the precise implicit bias can involve other Schatten-type norms and depends on the parameterization and initialization.

Proof. Sketch of the key ideas (full proof in Gunasekar et al. 2018):

Step 1: Gradient flow characterization

For factorized linear model $W = UV^T$ with gradient flow on U, V :

$$\begin{aligned} \frac{dU}{dt} &= (y - UV^T x)x^T V \\ \frac{dV}{dt} &= U^T (y - UV^T x)x \end{aligned} \quad (19)$$

This differs from direct gradient flow on W because of the factorization.

Step 2: Trajectory analysis

Key observation: Starting from small initialization $U_0, V_0 \approx 0$, the trajectory has a specific structure. Define $Z_t = U_t V_t^T$ (the product). One can show:

$$\frac{dZ_t}{dt} = \frac{dU_t}{dt} V_t^T + U_t \frac{dV_t^T}{dt} \quad (20)$$

After substitution and analysis, this simplifies to show the trajectory stays in a low-dimensional subspace.

Step 3: Implicit regularization

In the limit as $t \rightarrow \infty$ with infinitesimally small initialization:

- The solution approaches the direction of minimum norm that fits the data
- For linear networks: this is exactly the **nuclear norm** $\|W\|_* = \sum_i \sigma_i(W)$ (sum of singular values)
- Nuclear norm is the convex relaxation of rank minimization

Step 4: Connection to nuclear norm minimization

Via convex duality and careful limiting arguments, one shows:

$$\lim_{\varepsilon \rightarrow 0} \lim_{t \rightarrow \infty} W_t = \operatorname{argmin}\{\|W\|_* : Wx_i = y_i \text{ for all } i\} \quad (21)$$

Thus gradient descent implicitly minimizes nuclear norm—favoring low-rank solutions. ■

Example: Matrix Factorization: Explicit vs Implicit

Problem: Recover matrix M^* from linear measurements $\mathcal{A}(M) = b$.

Explicit regularization (convex optimization):

$$\min_M \|M\|_* \quad \text{subject to} \quad \mathcal{A}(M) = b \quad (22)$$

Requires solving SDP (slow).

Implicit regularization (gradient descent on factorization):

$$\min_{U,V} \|\mathcal{A}(UV^T) - b\|_2^2 \quad \text{starting from small initialization} \quad (23)$$

Gradient descent on U, V implicitly minimizes nuclear norm!

Why this matters: Non-convex optimization (factorization) can solve convex problem (nuclear norm) via implicit bias. Computation is much faster, and solution quality is often better.

 Proof Template: Analyzing Implicit Bias

To show gradient descent has implicit bias toward a specific regularizer:

Step 1: Characterize gradient flow

- Write continuous-time dynamics: $\frac{d\theta}{dt} = -\nabla \mathcal{L}(\theta)$
- Analyze trajectory starting from initialization θ_0

Step 2: Identify conserved quantities

- Find quantities preserved along gradient flow
- For linear networks: certain ratios or products of weight matrices

Step 3: Characterize limiting solution

- As $t \rightarrow \infty$, characterize $\lim_{t \rightarrow \infty} \frac{\theta_t}{\|\theta_t\|}$ (direction)
- Show it solves a specific optimization problem (e.g., max-margin, min nuclear norm)

Step 4: Verify implicit regularization

- Prove limiting solution minimizes explicit regularizer among all solutions with zero loss
- Connect to generalization bounds

Example: Logistic regression on separable data

- Gradient flow: $\frac{dw}{dt} = \sum_i y_i x_i \sigma(-y_i w^T x_i)$ where σ is sigmoid
- As $t \rightarrow \infty$: $\|w_t\| \rightarrow \infty$ but $\frac{w_t}{\|w_t\|} \rightarrow \frac{w_{\text{SVM}}}{\|w_{\text{SVM}}\|}$
- Implicit bias: max-margin solution!

Implicit Margin Maximization

Theorem: Implicit Margin Maximization (Soudry et al. 2018)

For linearly separable classification with logistic or exponential-train gradient-descent/gradient-flow conditions, the weight norm diverges and converges to the **max-margin** solution (the same solution found by SVM).

$$\lim_{t \rightarrow \infty} \frac{w_t}{\|w_t\|_2} = \frac{w_{\text{SVM}}}{\|w_{\text{SVM}}\|_2}.$$

In words: gradient descent on logistic loss implicitly maximizes the **margin** (the minimum distance from training points to the decision boundary), just like SVM—even though we never explicitly asked for max margin!

This explains why neural networks generalize well on separable data: they implicitly maximize margin, which classical theory (Week 2-3, margin bounds) shows leads to good generalization. The **margin** of a classifier is $\min_i y_i \frac{w^T x_i}{\|w\|}$ (the distance from the decision boundary to the nearest training point).

Example: Visualizing Implicit Bias

Setup: Train logistic regression on linearly separable 2D data.

Observations:

1. Training loss decreases exponentially fast initially
2. Training loss decreases linearly (in log scale) after separation
3. Weight norm $\|w_t\|$ grows unboundedly
4. Weight direction $\frac{w_t}{\|w_t\|}$ converges to SVM solution
5. Margin $\min_i y_i \frac{w_t^T x_i}{\|w_t\|}$ increases monotonically

Key insight: Even though we never explicitly asked for max margin, gradient descent finds it!

Example: Flat vs. Sharp Minima

Two solutions can have the same training loss but different generalization:

- **Sharp minimum:** Loss increases rapidly with small parameter perturbations → the solution is sensitive to noise → may generalize poorly
- **Flat minimum:** Loss remains stable under parameter perturbations → the solution is robust to noise → often correlates with better generalization

SGD with noise (from mini-batch sampling) can bias training toward wider basins (flatter minima), but naive sharpness is not invariant under reparameterization (e.g., rescaling weights). Rigorous statements usually use normalized sharpness or PAC-Bayes notions of flatness that account for parameterization invariance.

Key Takeaway

Gradient descent implicitly regularizes in several ways: low-rank/low-norm biases in some linear-network settings, max-margin direction in separable classification, and empirically flatter solutions in deep networks. These mechanisms help explain generalization without explicit regularization (no manual penalty terms like $\lambda \|\theta\|_2^2$ added to the loss), but their exact form depends on model architecture, loss function, initialization scheme, and optimizer choice.

Remark

The role of initialization is crucial for implicit regularization:

- **Small initialization:** Encourages low-norm solutions (parameters stay close to zero), producing stronger implicit bias toward simplicity
- **Xavier/Glorot initialization:** Preserves variance across layers (prevents activations from vanishing or exploding), helps prevent vanishing/exploding gradients
- **He initialization (for ReLU):** Accounts for ReLU killing half the neurons (sets variance to $\frac{2}{n_{\text{in}}}$ instead of $\frac{1}{n_{\text{in}}}$)

The initialization scale affects both optimization speed (too small $\rightarrow$ slow learning; too large $\rightarrow$ instability) and which solution gradient descent finds (different initializations lead to different implicit biases).

✓ Checkpoint

Implicit regularization summary:

1. Gradient descent + loss + initialization = implicit bias toward “simple” solutions
2. Linear networks: bias toward low nuclear norm (low rank)
3. Separable classification: bias toward max margin (like SVM)
4. Deep networks: bias toward flat minima (robust solutions)

Connection to generalization: These biases explain why networks with $p \gg n$ don't overfit!

Why Implicit Regularization Matters

Connection**Connecting to classical ML theory:**

Week 2-3 (PAC Learning): Sample complexity scales as $\tilde{O}(\frac{d}{\epsilon^2})$ where d is VC dimension (a measure of hypothesis class complexity).

- For neural networks: $d = O(p \log p)$ where p is number of parameters
- If $p \gg n$: classical theory predicts overfitting (the model has too much capacity)

Resolution via implicit regularization:

- Effective dimension is much smaller than p (not all parameters contribute equally to model complexity)
- Gradient descent searches only a small subspace of all possible parameter configurations (the optimization trajectory stays in a low-dimensional manifold)
- Implicit bias toward low-complexity solutions (gradient descent prefers simple solutions even without explicit penalties)
- Effective sample complexity depends on implicit regularization, not just raw parameter count p (what matters is the effective capacity, not the total number of parameters)

Key insight: Not all hypotheses with p parameters are equally likely under gradient descent. The optimization algorithm restricts the effective hypothesis class to a much smaller subset!

Connection

Preview of Week 12: We've seen implicit regularization exists (this week). Week 12 will:

1. Quantify implicit regularization (PAC-Bayes, compression bounds)
2. Connect to modern phenomena (grokking, double descent)
3. Analyze when implicit regularization succeeds vs fails
4. Discuss practical implications (architecture, optimization choices)

Wrong Path: More Parameters Always Means Overfitting

Classical intuition: More parameters $\rightarrow$ more capacity $\rightarrow$ overfitting

Why this is wrong for neural networks:

1. **Implicit regularization:** Optimization restricts effective capacity
2. **Overparameterization helps optimization:** More parameters $\rightarrow$ easier loss landscape
3. **Implicit bias:** Gradient descent finds “simple” solutions even with many parameters

Correct intuition: Parameters $\neq$ capacity. Effective capacity depends on optimization algorithm.

Analogy: A library with 1 million books has high potential capacity, but if you only read books recommended by a careful librarian (gradient descent), your effective exposure is much smaller—and potentially better curated.

Progress on backbone: We've now seen why overparameterized networks don't overfit despite having enough capacity to memorize—implicit regularization constrains the solution space. But classical learning theory predicts a monotonic relationship between model complexity and generalization error (the bias-variance tradeoff from Week 4). Modern deep learning exhibits something radically different: the **double descent** phenomenon, where increasing model size **beyond** the interpolation threshold actually **improves** generalization. This challenges everything we learned in the first half of the course.

Double Descent and Modern Phenomena

We now examine a surprising modern discovery that challenges classical learning theory.

Opening Challenge

Classical theory predicts U-shaped test error (the bias-variance trade-off from Week 4-5). Modern deep learning shows **double descent** (Belkin et al. 2019): error decreases (first descent), then increases sharply at the interpolation threshold ($p \approx n$), then decreases again (second descent)!

Connection

Connection to Week 4-5 (Bias-Variance Trade-off): Classical trade-off predicts:

- Low complexity: high bias (model too simple to capture patterns), low variance (stable across different training sets)
- High complexity: low bias (model flexible enough to fit patterns), high variance (sensitive to training data randomness)
- Optimal complexity: balance between bias and variance (minimizes total expected error)

Double descent extends this: **very** high complexity can reduce variance again via implicit regularization! Beyond the interpolation threshold, having more parameters gives the optimizer more choices, allowing it to find smoother solutions that happen to fit the data.

Definition: Three Regimes of Learning

As the number of parameters p increases relative to the number of training samples n :

1. **Underparameterized** ($p < n$): Cannot fit all training data perfectly; classical bias-variance trade-off applies
2. **Interpolation threshold** ($p \approx n$): Just enough parameters to fit training data perfectly; peak test error occurs here (worst generalization!)
3. **Overparameterized** ($p \gg n$): Far more parameters than needed to fit data; test error decreases with more parameters due to implicit regularization!

Example: Double Descent in Practice

Train networks of varying width on MNIST:

- Width 1-10: High test error (underparameterized)
- Width 20: Peak test error (interpolation threshold)
- Width 20-100: Test error decreases (overparameterized)

Two descents: before and after interpolation!

Mental Model: Understanding Double Descent

Classical regime (underparameterized):

- Model cannot fit all data points
- Increasing complexity: bias $\downarrow$, variance $\uparrow$
- Minimum test error at optimal complexity

Interpolation threshold:

- Model barely fits all data points (one solution or very few)
- Forced to fit noise exactly
- No room for implicit regularization
- Peak test error (worst case!)

Overparameterized regime:

- Many solutions fit training data perfectly
- Gradient descent can choose among them via implicit bias
- Finds “simplest” interpolating solution
- Test error decreases with more parameters!

Key insight: Having more parameters gives optimizer more choices $\rightarrow$ can find better (simpler) solutions that happen to fit the data.

Analogy: Imagine connecting n points with curves:

- Degree $< n$: cannot pass through all points (underfit)
- Degree $= n$: unique polynomial, very wiggly (forced interpolation)
- Degree $\gg n$: many polynomials work, can choose smoothest one (overparameterized)

Author's Take

Informal double-descent explanation:

Test error can be viewed as approximation error plus variance from fitting finite/noisy data.

- At threshold: the model may be forced to fit noise (high variance)
- Beyond threshold: the optimizer may have many interpolating solutions and find a smoother one through implicit regularization

This pattern is observed and proved in selected models, but it is not guaranteed for every architecture, dataset, and noise distribution.

Theorem: Double Descent for Linear Models (Hastie et al. 2019)**Assume:**

- **Model:** Linear regression $y_i = x_i^T \theta^* + \varepsilon_i$ with $\theta^* \in \mathbb{R}^p$ and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$
- **Data:** Features $x_i \sim \mathcal{N}(0, \Sigma)$ with covariance $\Sigma \in \mathbb{R}^{p \times p}$, eigenvalues $\lambda_1, \dots, \lambda_p$
- **Estimator:** Ridgeless least squares (minimum-norm interpolation)
 - If $p \leq n$: $\hat{\theta} = (X^T X)^{-1} X^T y$ (OLS, unique if full rank)
 - If $p > n$: $\hat{\theta} = X^T (X X^T)^{-1} y$ (minimum-norm solution)
- **Regime:** Vary p while holding n fixed

Then: The test risk $R(\hat{\theta}) = \mathbb{E}_{x,y} \left[(y - x^T \hat{\theta})^2 \right]$ exhibits double descent:

1. **Classical regime** ($p < n$): Risk decreases as p increases (more features $\rightarrow$ better fit)
2. **Interpolation peak** ($p = n$): Risk spikes (overfitting at threshold)
3. **Modern regime** ($p > n$): Risk decreases again as p increases (overparameterization helps!)

Risk formula (simplified):

$$R(\hat{\theta}) \approx \begin{cases} \sigma^2 \left(1 + \frac{p}{n}\right) & \text{if } p < n \\ \text{diverges} & \text{if } p = n \\ \sigma^2 \left(1 + \frac{n}{p}\right) & \text{if } p \gg n \end{cases} \quad (25)$$

The exact behavior depends on the eigenvalue spectrum of Σ .

Why does risk decrease for $p > n$?

Intuition: With many features ($p \gg n$), there are many interpolating solutions. Minimum-norm selection (implicit in ridgeless LS) chooses the smoothest, which generalizes better. This is implicit regularization from the optimization procedure.

Key Takeaway

Double descent extends the classical bias-variance story in some overparameterized settings. Bigger models can generalize better due to implicit regularization beyond the interpolation threshold, but validation remains necessary because the pattern is not universal (it depends on the data distribution, model architecture, and training procedure).

Thought Experiment

Imagine fitting a polynomial to noisy data:

Underparameterized (degree 2 polynomial for 10 data points):

- Smooth parabola that cannot fit all points exactly
- Moderate test error (high bias because parabola is too simple, low variance because the fit is stable)

Interpolation threshold (degree 10 polynomial for 10 data points):

- Unique wiggly polynomial passes through all points, forced to fit noise exactly
- High test error (low bias because it fits training data perfectly, high variance because it's forced to fit noise in a specific wiggly way)

Overparameterized (degree 100 polynomial for 10 data points):

- Many polynomials can fit the data perfectly (infinitely many solutions)
- Gradient descent finds the smoothest one (minimum-norm interpolator among all degree-100 polynomials)
- Lower test error (low bias because it interpolates, moderate variance because smoothness constraint reduces sensitivity to noise)

Key insight: having **more** parameters (degree 100 vs degree 10) gives the optimizer **more choices**, allowing it to find better solutions (smooth interpolators rather than wiggly forced fits)!

Example: Double Descent Across Multiple Axes

Double descent has been observed across multiple dimensions:

Model size (width of neural network layers):

- Peak test error at width $\approx n$ (interpolation threshold)
- Test error decreases for width $\gg n$ (overparameterized regime)

Training epochs (number of passes through the training data):

- Initially: test error decreases (learning the signal from data)
- Middle: test error increases (starting to fit noise in the data)
- Late: test error decreases again (implicit regularization takes over, finding smoother solutions)

Sample size (varying n with fixed model complexity):

- Peak test error when $n \approx p$ (model complexity matches sample size)
- For fixed p , increasing n past the threshold reduces error (more data helps beyond the critical point)

✓ **Checkpoint****Double descent insights:**

1. Classical U-shaped curve is incomplete—there's a second descent!
2. Interpolation threshold ($p \approx n$) is the worst regime
3. Overparameterization ($p \gg n$) can improve generalization via implicit regularization
4. Pattern observed across models (linear, neural networks, trees) but not universal

Key question: When does double descent occur? When does it fail?

When Does Double Descent Occur?

Author's Take**Conditions favoring double descent (when we expect to see the phenomenon):**

1. **Interpolation regime:** Model must be able to fit training data perfectly (zero training loss achievable)
2. **Multiple solutions:** Many parameter settings achieve zero training loss (gives optimizer room to choose)
3. **Implicit regularization:** Optimization algorithm has bias toward “simple” solutions (e.g., gradient descent prefers low-norm solutions)
4. **Benign overfitting:** Fitting noise doesn't hurt too much in the overparameterized regime (noise is absorbed without affecting generalization much)

Conditions working against double descent (when the phenomenon may not appear):

1. **Explicit regularization:** Strong ℓ_2 penalty ($\lambda \|\theta\|_2^2$ added to loss) prevents interpolation (forces training error above zero)
2. **Early stopping:** Stopping training before interpolation avoids both the interpolation threshold peak and the second descent
3. **Strongly regularized tasks:** When explicit regularization dominates implicit bias (e.g., very large λ)
4. **Adversarial robustness requirements:** Interpolating solutions can be non-robust to adversarial perturbations (small worst-case input changes that fool the model)

Wrong Path: Always Use Maximally Overparameterized Models

Tempting conclusion: If bigger is better in overparameterized regime, always use huge models!

Why this is wrong:

1. **Computational cost:** Training scales with parameters
2. **Not always better:** Double descent isn't universal
3. **Diminishing returns:** Gains plateau eventually
4. **Robustness concerns:** Interpolating solutions can be non-robust
5. **Implicit regularization can fail:** Not all overparameterized models generalize

Better approach: Use validation to find appropriate model size. Overparameterization helps but isn't a panacea.

Connection

Course synthesis: Week 11 integrates concepts from all previous weeks:

- **Mathematical foundations (Week 1):** Convexity, Hessian matrices, concentration inequalities
- **PAC learning (Weeks 2-3):** Sample complexity, approximation-estimation trade-off
- **Model selection (Weeks 4-5):** Bias-variance trade-off extended to double descent
- **Kernel methods (Weeks 6-8):** NTK connects networks to kernel methods
- **Optimization (Weeks 9-10):** Gradient descent convergence, online learning

Central question answered: Why do overparameterized networks generalize?

- Overparameterization helps optimization (creates benign loss landscapes with few bad local minima)
- Implicit regularization prevents overfitting (gradient descent has bias toward simple solutions)
- Double descent: beyond interpolation threshold, more parameters help via implicit regularization!

Connection

Preview of Week 12: We've established:

- Networks can represent functions (expressivity)
- Networks can be trained (optimization)
- Networks have implicit biases (regularization)

Week 12 addresses remaining questions:

- How to quantify implicit regularization? (PAC-Bayes, compression)
- What are modern phenomena? (Grokking, scaling laws, emergence)
- When does theory match practice? (Realistic models)
- How to design better architectures? (Practical implications)

The Big Picture: Three Puzzles Resolved

Backbone resolution: Having explored expressivity, optimization, kernel theory, implicit regularization, and double descent, we can now directly answer our central question: **Why do overparameterized neural networks generalize?** The answer synthesizes everything we've learned this week.

Mental Model: Resolution of the Three Puzzles

Recall the three puzzles from the introduction. Here's how this week's theory resolves them:

Puzzle 1: Why doesn't gradient descent get stuck in local minima?

- Resolution: Overparameterization creates benign landscapes (loss surfaces that are relatively smooth and easy to optimize)
- Most critical points are saddles, not local minima (in high dimensions, having all Hessian eigenvalues positive is rare)
- SGD noise helps escape saddles (stochastic gradient descent's randomness pushes parameters away from saddle points)
- Very wide networks satisfy PL condition $\rightarrow$ exponential convergence (loss decreases exponentially fast despite non-convexity)

Puzzle 2: Why don't overparameterized networks overfit?

- Resolution: Implicit regularization from gradient descent (the optimization algorithm itself prefers certain solutions)
- Bias toward low-norm, low-rank, or max-margin solutions (gradient descent finds "simple" solutions among many possibilities)
- Effective capacity much smaller than parameter count (not all parameters are used equally)
- Optimization algorithm restricts hypothesis class (gradient descent explores only a small subspace of all possible parameter settings)

Puzzle 3: Why does interpolation (zero training error) not hurt?

- Resolution: Many interpolating solutions exist; gradient descent finds good ones (among all perfect-fit solutions, it chooses smooth ones)
- Implicit bias toward "simple" interpolators (low-norm or max-margin solutions that happen to fit the data)
- Double descent: beyond threshold, more parameters help (gives optimizer more choices to find better solutions)
- Benign overfitting in overparameterized regime (fitting noise doesn't significantly hurt test performance)

The unified story: Overparameterization helps both optimization (easier to find good solutions) AND generalization (implicit bias finds solutions that generalize well)!

Practice Problems

Conceptual Understanding (★)

1. Why doesn't universal approximation guarantee practical success?

Hint: Think about what the theorem provides (existence) vs what we need (construction, sample complexity, computation).

2. Explain why deep networks are exponentially more efficient than shallow networks for compositional functions.

Hint: Consider how many features shallow networks need to enumerate vs how deep networks build hierarchically.

3. What's the difference between local minima and saddle points? Why aren't saddles problematic in high dimensions?

Hint: Count constraints: to be a local minimum, ALL Hessian eigenvalues must be positive. In high dimension, this is rare.

4. Distinguish explicit regularization (weight decay) from implicit regularization (gradient descent bias).

Hint: Explicit: added to objective function. Implicit: emergent property of optimization algorithm.

5. In the NTK regime, why does training become linear?

Hint: Features stay constant at initialization, only coefficients change—exactly like kernel methods!

6. Explain double descent in one sentence.

Hint: Test error peaks at interpolation threshold, then decreases again in overparameterized regime.

Theoretical Analysis (★★)

7. Prove ReLU networks can approximate any continuous piecewise linear function exactly.

Hint: Each ReLU creates one “bend”. Show how to combine them to match all breakpoints.

8. For two-layer linear network $f(x) = W_2 W_1 x$, show all critical points with $W_2 W_1 = W^*$ are global minima.

Hint: Loss is convex in product $W_2 W_1$. Any factorization with correct product has same loss.

9. Compute the NTK for a two-layer ReLU network: $f(x) = \left(\frac{1}{\sqrt{m}}\right) \sum_{i=1}^m a_i \max(0, w_i^T x)$.

Hint: Take gradient w.r.t. parameters, compute inner product. Use law of large numbers as $m \rightarrow \infty$.

10. Prove exponential convergence under the PL condition.

Hint: Use smoothness to get descent lemma, then apply PL to bound gradient norm by suboptimality.

11. Show that linear regression satisfies the PL condition when X has full rank.

Hint: Express gradient in terms of residual, use smallest eigenvalue of $X^T X$.

12. Verify that the NTK training dynamics are linear: $\frac{d\mathbf{f}_t}{dt} = -\frac{1}{n}\Theta(\mathbf{f}_t - \mathbf{y})$.

Hint: Chain rule: $\frac{df_t(x)}{dt} = \nabla_{\theta} f(x; \theta_t)^T \frac{d\theta_t}{dt}$. Use gradient flow for parameters.

Advanced Theory (★★★)

13. Prove Barron's theorem: functions with bounded Fourier spectrum achieve $O\left(\frac{1}{\sqrt{n}}\right)$ approximation error.

Hint: Use random feature approximation: sample projections according to Fourier spectrum, get Monte Carlo error rate.

14. Show the NTK remains constant during training in the infinite-width limit.

Hint: Each neuron's contribution scales as $\frac{1}{\sqrt{m}}$, so parameters move $O\left(\frac{1}{\sqrt{m}}\right)$. Gradient norm is $O(1)$, so NTK change is $O\left(\frac{1}{\sqrt{m}}\right)$.

15. Prove gradient descent on logistic loss converges in direction to the max-margin solution.

Hint: Show $\|w_t\| \rightarrow \infty$ but $\frac{w_t}{\|w_t\|}$ converges. Analyze limiting direction as minimizer of norm subject to correct classification.

16. Prove that deep linear networks with squared loss have no bad local minima under appropriate rank assumptions.

Hint: Show loss is convex in end-to-end mapping. Use Morse theory or direct analysis of critical points.

Computational Experiments (🤔)

17. Visualize loss landscapes for networks of varying width. Compare narrow vs. wide.

Hint: Fix two random directions in parameter space, plot loss as function of (α, β) . Observe: wider $\rightarrow$ smoother.

18. Train networks of varying width on a small dataset. Plot test error vs. parameters. Observe double descent?

Hint: Try MNIST subset with $n = 100$ samples. Vary width from 5 to 1000. Plot train/test curves.

19. Train a two-layer linear network. Does gradient descent find minimum nuclear norm or minimum Euclidean norm?

Hint: Initialize small. Train to zero loss. Measure nuclear norm $\|W_2 W_1\|_$ vs Frobenius norm $\|W_2\|_F^2 + \|W_1\|_F^2$. Compare to explicit nuclear norm minimization.*

20. Track margin during training on separable data. Does it increase monotonically? Compare to SVM.

Hint: Train logistic regression on separable 2D data. Plot margin $\min_i y_i \frac{w_i^T x_i}{\|w_i\|}$ vs iteration. Compare final direction to SVM solution.

21. Implement a very wide network and verify lazy training regime (compare to kernel ridge regression with NTK).

Hint: Train width-10000 network with small learning rate. Compute NTK at initialization. Compare predictions to kernel ridge regression with same kernel.

22. Visualize feature learning vs lazy training. Compare moderate width to very large width.

Hint: Track $\frac{\|\theta_t - \theta_0\|}{\|\theta_0\|}$ during training. Lazy regime: stays small. Feature learning: grows significantly.

Synthesis and Connections (🤔)

23. Compare NTK to classical kernel methods (Week 11 Background Section). Advantages and disadvantages?

Hint: NTK advantages: derived from architecture, adapts to task (at initialization). Disadvantages: lazy training limits representation, fixed after initialization. Classical kernels: designed for task, but no representation learning.

24. How does double descent relate to bias-variance trade-off (Week 4-5)? Contradiction or extension?

Hint: Extension! Classical: one descent. Modern: two descents with peak at interpolation threshold. Implicit regularization enables second descent.

25. Compare gradient descent convergence in: (a) convex optimization, (b) non-convex with PL, (c) overparameterized networks.

Hint: (a) Convex + smooth: $O(\frac{1}{T})$ to first-order stationary point; strongly convex: exponential. (b) PL: exponential convergence despite non-convexity. (c) Very wide: PL-like behavior via NTK; moderate width: empirically fast but less theory.

26. Design an experiment to test whether SGD and Adam have different implicit biases.

Hint: Train same architecture with SGD vs Adam on separable data. Compare learned solutions: margins, norms, flatness. Which finds “simpler” solution?

27. How does implicit regularization connect to PAC learning theory (Week 2-3)?

Hint: PAC: sample complexity depends on hypothesis class complexity. Implicit regularization: optimization restricts effective hypothesis class, reducing effective complexity below $O(p)$.

28. Why does overparameterization help both optimization AND generalization? Isn't this contradictory?

Hint: Not contradictory! More parameters $\rightarrow$ easier optimization (more degrees of freedom to avoid bad minima). Implicit bias $\rightarrow$ effective capacity stays controlled. Having many parameters $\neq$ using all of them.

29. Connect depth separation (expressivity) to sample complexity (learnability).

Hint: Approximation-estimation trade-off: better approximation (deep networks) $\rightarrow$ smaller approximation error $\rightarrow$ better sample complexity. But must also control estimation error via implicit regularization.

30. Critique: "NTK theory proves deep learning works." What's missing?

Hint: NTK explains: optimization is easy (lazy regime). NTK misses: (1) feature learning (finite width), (2) why learned features generalize, (3) architecture design, (4) practical training dynamics. NTK is one piece, not the full story!

✓ Checkpoint

Problem-solving strategies:

1. Conceptual: build intuition from mental models, connect to previous weeks
2. Theoretical: use proof templates (PL condition, loss landscape, NTK dynamics)
3. Computational: implement small-scale experiments to verify theory
4. Synthesis: connect concepts across weeks, identify gaps between theory and practice

These problems prepare you for Week 12's deeper dive into generalization theory and modern phenomena.

Summary

Key Takeaway

Week 11 main insights:

1. **Expressivity (Week 10):** Universal approximation guarantees existence of solutions; depth separation shows compositional efficiency. This week addresses: how does optimization find these solutions, and why do they generalize?
2. **Optimization landscape:** Overparameterized networks often have more benign landscapes under suitable assumptions, with fewer bad local minima in many models. High-dimensional parameter spaces make it hard for critical points to be local minima rather than saddles.
3. **Neural Tangent Kernel:** Infinite-width networks behave like kernel methods with a specific kernel determined by architecture and initialization, explaining easy optimization in the lazy training regime (where features stay frozen at initialization).
4. **Implicit regularization:** Gradient descent can prefer low-norm, low-rank, or max-margin solutions, aiding generalization in model-dependent ways. This emerges from the optimization dynamics, not from explicit penalty terms in the loss.
5. **Double descent:** Beyond the interpolation threshold ($p \approx n$), bigger models generalize better via implicit regularization. This extends the classical U-shaped bias-variance curve with a second descent.

 Mental Model: The Complete Week 11 Story

The central question: Why do overparameterized neural networks generalize?

Part 1: Expressivity (What can networks represent?)

- Universal approximation: any continuous function (existential result)
- Depth vs width: deep networks exponentially more efficient for compositional functions
- Connection to sample complexity via approximation-estimation trade-off

Part 2: Optimization (How do we train networks?)

- Non-convex optimization is hard in general
- Overparameterization creates benign loss landscapes
- High dimension helps: most critical points are saddles
- PL condition gives exponential convergence without convexity

Part 3: NTK regime (When are networks like kernels?)

- Infinite width $\rightarrow$ lazy training $\rightarrow$ kernel methods
- Training dynamics become linear
- Explains optimization but misses feature learning
- Practical networks (finite width) learn features

Part 4: Implicit regularization (Why don't networks overfit?)

- Gradient descent + loss + initialization = implicit bias
- Linear networks: low nuclear norm (low rank)
- Separable classification: max margin (like SVM)
- Deep networks: flat minima, “simple” solutions

Part 5: Double descent (Modern phenomena)

- Classical bias-variance: one descent
- Modern: second descent beyond interpolation threshold
- Overparameterization helps via implicit regularization
- Pattern observed but not universal

The unified answer: Overparameterization helps both optimization (benign landscape) AND generalization (implicit regularization). The optimization algorithm restricts the effective hypothesis class, preventing overfitting despite many parameters.

Author's Take

Week 11 answers: “Can we train neural networks?” Yes—overparameterization makes optimization tractable even though the problem is non-convex.

Week 12 will answer: “Why do trained networks generalize?” We’ll explore implicit regularization more deeply, compression bounds, margin theory, and modern phenomena (grokking, scaling laws, emergence).

What we’ve learned:

- Theory is catching up to practice (deep learning worked before we understood why)
- Overparameterization is a feature, not a bug (more parameters help both optimization and generalization)
- Implicit regularization is crucial (the optimization algorithm restricts what the model learns)
- Classical theory (VC dimension, PAC bounds) is incomplete for deep learning (these bounds are too pessimistic)

What remains mysterious:

- Exact characterization of implicit bias for deep nonlinear networks (we understand linear networks better)
- When does implicit regularization succeed vs fail? (what properties of data/architecture matter?)
- How to design architectures for better implicit bias? (can we engineer networks to have better inductive biases?)
- Connection between optimization and generalization (why does SGD generalize better than Adam or full-batch gradient descent?)

These questions motivate Week 12’s deeper exploration of generalization theory.

Connection

Preparation for Week 12:

Week 11 established the foundations:

- Networks can represent functions (expressivity)
- Networks can be optimized (landscape, PL condition, NTK)
- Networks have implicit biases (regularization)

Week 12 will build on these foundations:

- Quantifying implicit regularization (PAC-Bayes, compression)
- Modern phenomena (grokking, scaling laws, emergence, cata)
- Practical implications (architecture design, optimization cho
- Open problems and future directions

Key concepts to remember:

1. Overparameterization helps optimization and generalization
2. Implicit regularization explains why $p \gg n$ doesn't cause overfitting
3. NTK connects networks to kernels (but misses feature learning)
4. Double descent extends bias-variance trade-off
5. Classical theory incomplete—need new frameworks

These concepts form the backbone for understanding modern deep learning theory!

Historical note: The theory-practice gap in deep learning has narrowed significantly since 2012:

- 2012: AlexNet wins ImageNet (empirical success)
- 2016: Depth separation theorems (expressivity)
- 2018: NTK theory (optimization)
- 2018: Implicit margin maximization (generalization)
- 2019: Double descent (modern phenomena)
- 2020-present: Scaling laws, grokking, emergence

Theory evolves to explain practice, which guides new theory. Week 12 covers the latest developments!